Génération d'intentions en langage naturel pour l'offloading piloté par l'intention

Méthodologie suivie de bout en bout, fondements dans la littérature, exemples détaillés et protocole de validation proposé

Jeu ARVR-150 — 150 scénarios, 150 intentions

Octobre 2026

Résumé

Ce document décrit comment les 150 intentions du jeu ARVR-150 ont été produites, pourquoi chaque choix a été fait, ce que la littérature apporte (et n'apporte pas), et comment ces intentions devraient être validées avant d'être utilisées comme jeu d'évaluation. Il distingue systématiquement ce qui a été **fait** de ce qui est seulement **proposé, non exécuté**. Chaque intention a été rédigée *à la main* par un LLM (Claude), scénario par scénario, sans aucun algorithme de composition de texte ; le seul code utilisé affiche les exigences sans la vérité terrain, contrôle mécaniquement le texte écrit et l'enregistre. Aucune relecture humaine n'a eu lieu : le statut des 150 entrées est `llm_generated_auto_checked`, jamais `validated`.

Table des matières

1 Objectif et vue d'ensemble **2**

1.1 Ce qu'on cherche à évaluer . . . 2

1.2 Ce qu'il faut produire . . . 2

1.3 Principe directeur . . . 2

2 Données d'entrée **2**

2.1 Le modèle applicatif ARVR . . . 2

2.2 Les exigences d'un scénario . . . 2

2.3 Ce que le rédacteur a le droit de voir . . . 3

3 Ancrage dans la littérature **3**

3.1 Six articles, un cadre de contrôle . . . 3

3.2 Ce que la littérature ne couvre pas . . . 3

4 Méthode de bout en bout **4**

4.1 Règle d'or . . . 4

4.2 Le cycle par scénario . . . 4

4.3 Les quatre niveaux d'explicitation . . . 5

4.4 Règles de fond appliquées à chaque intention . . . 5

4.5 Diversité . . . 6

4.6 Les contrôles mécaniques 6

5 Exemples de bout en bout **6**

5.1 Exemple A — une exigence, niveau 4 (ARVR-000146) 6

5.2 Exemple B — quatre exigences, niveaux mêlés (ARVR-000138) 8

5.3 Un refus du contrôle (ARVR-000036) 8

6 Résultat de la génération **9**

6.1 Limites connues 9

7 Méthode de validation proposée **9**

7.1 Couche 1 — Reconstruction à l'aveugle (inspirée de Liu et al.) 9

7.2 Couche 2 — Revue humaine en double aveugle 10

7.3 Couche 3 — Sous-ensemble rédigé par des humains 11

7.4 Couche 4 — Fuite de décision et artefacts 11

7.5 Couche 5 — Diversité 11

7.6 Critères d'acceptation 11

7.7 Ce que cette validation ne prouve pas 11

8 Fichiers livrés **12**

1 Objectif et vue d'ensemble

1.1 Ce qu'on cherche à évaluer

Le pipeline étudié transforme une demande exprimée en langage naturel en une décision de placement de composants applicatifs sur une infrastructure :

intention (texte) $\longrightarrow$ exigences structurées $\longrightarrow$ décision $\in \{\text{STAY}, \text{OFFLOAD}, \text{INFEASIBLE}\}$.

La première étape (le *Requirement Extractor*, actuellement `openai/gpt-5.4-mini` via OpenRouter) lit l'intention et en tire des exigences : pour chacune, une cible (composant `C1`–`C5` ou flux `f1`, `f2`), un indicateur (`cpu`, `ram`, `e2e_latency`), un comparateur, une valeur et une unité. Le jeu d'intentions sert à mesurer *cette* étape : l'extracteur retrouve-t-il, à partir du texte seul, les exigences de référence ?

1.2 Ce qu'il faut produire

Pour chacun des 150 scénarios simulés de `eval/experiment_150/scenarios.json`, une intention en anglais qui :

- exprime **toutes** les exigences du scénario et **seulement** celles-ci ;
- donne la **valeur exacte** (aucun arrondi, aucun « environ ») ;
- soit **variée et humaine** : tantôt explicite, tantôt implicite ou vague *sur la forme* ;
- ne **révèle jamais la décision** attendue ni ne parle de placement.

1.3 Principe directeur

Le flou porte sur la forme, jamais sur le fond

Une intention peut désigner un composant par son rôle (« ce qui dessine les éléments de réalité augmentée »), omettre son identifiant, changer d'unité ou raconter un contexte. Elle ne doit jamais rendre la *valeur* approximative, ni rendre la *cible* ambiguë. Ce qui est difficile pour l'extracteur doit l'être parce qu'il faut *raisonner*, pas parce que la réponse est indécidable.

2 Données d'entrée

2.1 Le modèle applicatif ARVR

Application « Interactive Augmented Reality Application », 5 composants et 2 flux (tableaux 1 et 2).

Point critique. `C1`, `C3` et `C5` appartiennent aux deux flux. Seuls `C2` (reconnaissance) distingue `f1` et `C4` (rendu) distingue `f2`. Toute désignation indirecte d'un flux doit s'appuyer sur l'un de ces deux composants.

2.2 Les exigences d'un scénario

Chaque scénario contient une à quatre exigences (315 au total : 39 scénarios à une exigence, 77 à deux, 14 à trois, 20 à quatre). Une exigence est un tuple *(cible, indicateur, comparateur, valeur de base, nouvelle valeur, unité)*, avec trois indicateurs :

TABLE 1 – Composants (extraits de `services.json`).

Id	Nom	Description officielle
C1	Vision Capture	Captures visual inputs from the user device
C2	Object Recognition	Detects and identifies objects in captured frames
C3	Scene Processing	Processes visual and scene-context information
C4	AR Rendering	Produces augmented-reality rendering elements
C5	Display Output	Displays processed results to the receiving device

TABLE 2 – Flux (extraits de `flows.json`).

Id	Nom	Chemin	Description officielle
f1	Object Recognition Flow	C1 → C2 → C3 → C5	Object recognition and scene-processing results delive
f2	AR Rendering Flow	C1 → C3 → C4 → C5	Scene processing and AR rendering delivered to the d

Indicateur	Cible	Comparateur / unité canonique
`cpu`	composant	≥ (`gte`), vCPU
`ram`	composant	≥ (`gte`), GB
`e2e_latency`	flux	≤ (`lte`), ms

2.3 Ce que le rédacteur a le droit de voir

Les scénarios contiennent aussi la vérité terrain de la décision (catégorie, niveau de difficulté, décision attendue, placements faisables, nombre minimal de migrations, ...). Si le rédacteur la voit, le ton ou l'urgence peut la trahir, et l'évaluation devient circulaire. Les intentions ont donc été rédigées **uniquement** à partir de :

- l'identifiant du scénario et ses exigences (commande `intent_store.py view`, qui masque tout le reste) ;
- les noms et descriptions des composants et des flux.

Le fichier `scenarios.json` n'a pas été ouvert pour écrire.

3 Ancrage dans la littérature

3.1 Six articles, un cadre de contrôle

Les six articles du dépôt portent sur la génération de texte à partir de données (*data-to-text*, *table-to-text*). Notre tâche en est l'inverse sémantique : partir d'exigences structurées pour produire un texte dont on doit pouvoir retrouver ces exigences. Les erreurs qu'ils étudient (valeur fausse, mauvaise entité, information ajoutée) sont exactement celles qui invalideraient une intention.

3.2 Ce que la littérature ne couvre pas

Aucun de ces articles ne traite l'intention *vague ou implicite*. Dans ces travaux, une information « non vérifiable » est une *faute*. Ici, nous la produisons volontairement, mais **uniquement sur la forme** (désignation, vocabulaire, structure, contexte), jamais sur la valeur. C'est une adaptation

TABLE 3 – Ce que le protocole retient de chaque source.

Source	Ce qui est repris dans notre méthode
Sundararajan, Sripada & Reiter (NAACL 2024)	Entrée *atomique* et sémantiquement nommée : chaque exigence est présentée isolément, avec ses unités explicites ; l'ordre de présentation est mélangé pour éviter le biais du premier élément cité.
Chen et al. (EMNLP 2022)	Compréhension de la structure d'entrée, puis *délibération* : générer, relire, corriger. Origine de nos deux passes (rédaction puis contrôle).
Zhao & Sun (*Knowledge-Based Systems* 304, 2024)	*Planifier avant de générer* : décider quelles informations, dans quel ordre, avec quelle structure, avant d'écrire. Origine de l'étape de plan par scénario.
Thomson et al. (INLG 2023)	Vues sur les données et schéma de document ; *taxonomie d'erreurs* (nombre, nom, contexte, ajout) que nous adaptons en contrôles.
Kasner & Dušek (ACL 2024)	La diversité doit venir d'un *plan* explicite et pas du hasard d'échantillonnage ; ne pas laisser des exemples fuiter dans la sortie ; *documenter* que le modèle générateur diffère du modèle évalué.
Liu, Zheng & Kong (ACL 2025)	Générer puis *vérifier* par un contrôle d'implication (le texte entraîne-t-il bien l'information source ?). Origine de notre test de reconstruction à l'aveugle (section 7).

propre au projet, pas un résultat de la littérature. Le découpage en quatre niveaux d'explicitation (section 4.3) est également un choix de conception, non emprunté.

4 Méthode de bout en bout

4.1 Règle d'or

Interdiction d'un générateur automatique

Aucun gabarit, aucune table de phrases à trous, aucun tirage aléatoire de formulations, aucune boucle qui compose du texte. Chaque intention est écrite par jugement, après lecture du scénario. Le script `intent_store.py` ne choisit *aucun mot* : il affiche, vérifie et enregistre.

4.2 Le cycle par scénario

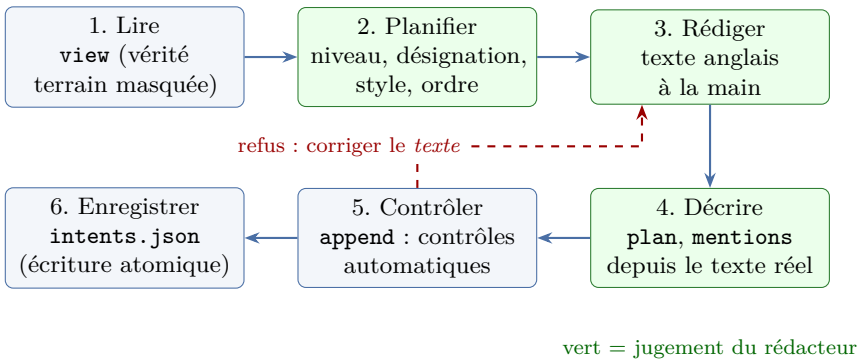


FIGURE 1 – Cycle de traitement d'un scénario. Les cases vertes sont des décisions du rédacteur ; les autres sont mécaniques.

Étape 1 — Lire

`python3 intent_store.py view next 5` affiche pour chaque scénario son identifiant et ses exigences (cible, indicateur, comparateur, valeur de base, nouvelle valeur, unité). Le *sens* de la variation se lit en comparant base et nouvelle valeur : CPU et RAM montent toujours, la latence peut se resserrer ou se relâcher.

Étape 2 — Planifier

Pour chaque exigence : niveau d'explicitation, mode de désignation de la cible, unité écrite. Pour l'ensemble : style, registre, structure, ordre de mention, longueur. Le plan est choisi *après* consultation de `intent_store.py report` (répartition courante) pour prendre volontairement ce qui est sous-représenté.

Étape 3 — Rédiger

En anglais, comme l'écrirait une personne réelle : ticket, message à un collègue, note d'exploitation, demande polie, remarque en revue. Pas de recopie des intentions voisines.

Étape 4 — Décrire

Les métadonnées (`plan`, `mentions`) sont écrites *après* le texte, à partir du texte réel, jamais l'inverse. Elles déclarent pour chaque exigence la forme écrite (`written_form`), la valeur et l'unité telles qu'écrites, le mode de désignation et le niveau.

Étapes 5 et 6 — Contrôler et enregistrer

Chaque scénario est contrôlé puis écrit *séparément* dans `intents.json` par écriture atomique (fichier temporaire puis renommage). Un refus n'efface jamais les entrées précédentes. Après toute coupure, `status` indique la prochaine entrée à traiter : la reprise est toujours possible, aucun travail n'est perdu.

4.3 Les quatre niveaux d'explicitation

Un niveau est attribué *par exigence*. Plusieurs niveaux coexistent dans un même scénario.

Répartition visée : 20 % / 30 % / 30 % / 20 %. Le niveau 1 contient un identifiant ; les niveaux 3 et 4 n'en contiennent jamais.

4.4 Règles de fond appliquées à chaque intention

1. **Toutes les exigences, et seulement celles-ci.** Aucun autre indicateur (bande passante, fiabilité, énergie…).
2. **Valeur exacte.** Reprise à toutes ses décimales ou convertie exactement : 1 vCPU = 1000 mCPU, 1 GB = 1000 MB, 1 s = 1000 ms. Unités binaires (GiB, MiB) écartées : leur conversion n'est pas décimale finie. Pas de séparateur de milliers.
3. **Comparateur déductible** : minimum pour CPU et RAM, maximum pour la latence.
4. **Cible unique.** Un lecteur qui ne connaît que les noms et descriptions doit retrouver sans hésiter le composant ou le flux. Un composant n'a pas de latence de bout en bout ; un flux n'a pas de CPU.

TABLE 4 – Niveaux d'explicitation (valeurs de l'exemple fictives).

Niv.	Cible désignée par	Indicateur et comparateur	Exemple
1	identifiant (`C2`, `f1`), avec ou sans nom	unités canoniques, comparateur en clair	« Keep the end-to-end latency of flow f2 at or below 412.5 ms. »
2	nom du composant ou du flux	indicateur nommé, unité variée	« The AR Rendering component needs at least 5250 millicores. »
3	responsabilité ou flux métier, sans nom ni id	indicateur reformulé, comparateur porté par la tournure	« Whatever draws the AR elements should get a minimum of 5.25 virtual CPUs. »
4	raisonnement sur les descriptions (position, conséquence, circonlocution)	indicateur nommé indirectement, registre familier	« The stage that turns detections into overlays is starving : 5.25 virtual CPUs minimum. »

5. **Application déjà déployée.** L'intention exprime une *évolution* de besoin.
6. **Cohérence de direction.** « tighten » seulement si la latence diminue, « relax » seulement si elle augmente.
7. **Interdits.** Mots de placement ou de mouvement (move, migrate, offload, place, host), identifiants de nœud, couches (edge, cloud, fog), disponibilité, fiabilité, faisabilité.
8. **Pas de fuite de décision.** Urgence et ton indépendants de la difficulté du scénario.
9. **Pas de nombre en trop.** Les seuls nombres sont les valeurs cibles (et les valeurs de base déclarées).

4.5 Diversité

La diversité est obtenue par le *plan*, pas par le hasard (Kasner & Dušek). Sont variés : style (impératif, demande polie, besoin, contrainte, objectif, récit, note de ticket, question), registre (développeur, manager, opérations télégraphiques, formel, familier, responsable produit), structure (une phrase, deux, conditionnelle, énumération, enchaînement), ordre de mention, longueur (7 à 57 mots) et ouverture. La variation doit être *structurelle* : remplacer « increase » par « raise » ne compte pas.

4.6 Les contrôles mécaniques

`append` refuse une entrée si l'une des conditions suivantes est violée (taxonomie adaptée de Thomson et al.) :

Règle de correction : en cas de refus, on corrige le *texte*, jamais le contrôle. Trois refus ont eu lieu, tous des faux positifs lexicaux (« three *places* », « ceiling *moves* », « all *around* »), corrigés par reformulation.

5 Exemples de bout en bout

5.1 Exemple A — une exigence, niveau 4 (ARVR-000146)

1. Lecture (sortie de `view`, vérité terrain masquée)

```
## ARVR-000146
  - flow f2 (AR Rendering Flow): e2e_latency lte 381.497 ms   [valeur de base 500]
```

TABLE 5 – Contrôles automatiques et type d'erreur de la taxonomie visé.

Contrôle	Erreur visée
Chaque exigence du scénario apparaît exactement une fois, aucune ajoutée	omission / ajout
Comparateur identique à la référence	contexte
Valeur écrite × facteur d'unité = valeur canonique, en arithmétique décimale exacte	nombre
La forme écrite figure dans le texte ; aucun nombre non déclaré	nombre / ajout
Identifiant présent si et seulement si la désignation le prévoit	nom
Aucun identifiant de nœud, mot de placement, couche, disponibilité, faisabilité	fuite / hors périmètre
Aucun mot d'approximation, tilde, séparateur de milliers, MIPS	nombre
Verbe de direction cohérent avec base → nouvelle valeur	contexte
Aucune intention en double ; avertissement si ouverture répétée ou longueur atypique	diversité

2. Plan La cible est le flux `f2`. On choisit le niveau 4 : ni nom ni identifiant. Il faut une périphrase qui désigne `f2` et *pas* `f1`. Les deux flux partagent la capture (`C1`) et l'affichage (`C5`), donc « de la caméra à l'écran » conviendrait aux deux et serait proscrit. Ce qui distingue `f2`, c'est le **rendu** (`C4`), qui produit des éléments de réalité augmentée : on parle donc de « visible overlay ». Style *question*, registre *casual*, structure *deux phrases*, valeur placée en fin (« tops » porte le « au plus »). La valeur passe de 500 à 381.497 : c'est un resserrement, donc aucun verbe de relâchement.

3. Texte

> *How quickly does a captured frame have to turn into a visible overlay? 381.497 ms, tops.*

4. Description


```
{"scenario_id": "ARVR-000146",
 "intent": "How quickly does a captured frame have to turn into a visible overlay? 381.497 ms,
    tops.",
 "plan": {"style": "question", "register": "casual", "structure": "two_sentences"},
 "mentions": [{"target_id": "f2", "kpi_type": "e2e_latency", "comparator": "lte",
   "written_value": "381.497", "written_unit": "ms", "written_form": "381.497 ms",
   "designation": "context", "level": 4}]}
```


5. Contrôles

Contrôle	Résultat
couverture : 1 exigence attendue, 1 déclarée	ok
comparateur `lte` = référence	ok
381.497 × 1 = 381.497 ms (valeur canonique)	ok
« 381.497 ms » présent dans le texte, aucun autre nombre	ok
désignation sans `id` ⇒ aucun `f1`/`f2` dans le texte	ok
aucun mot interdit, aucune approximation	ok
aucun verbe de direction (aucune incohérence possible)	ok

6. Ce que le contrôle ne dit pas. Que « visible overlay » désigne bien `f2` et non `f1` n'est **pas** vérifié par le code. C'est une affirmation de jugement, à confirmer par la validation sémantique (section 7).

5.2 Exemple B — quatre exigences, niveaux mêlés (ARVR-000138)

1. Lecture

```
## ARVR-000138
  - service C4 (AR Rendering): cpu gte 5.150654 vCPU         [valeur de base 3.364]
  - service C3 (Scene Processing): ram gte 4.454570016 GB    [valeur de base 2.9455208]
  - flow f1 (Object Recognition Flow): e2e_latency lte 511.645 ms  [valeur de base 500]
  - flow f2 (AR Rendering Flow): e2e_latency lte 506.74 ms         [valeur de base 500]
```

2. Plan Les deux latences *montent* (500 → 511.645 et 506.74) : il s'agit d'un relâchement, le verbe « relaxed » est permis. Les ressources sont des hausses. Choix :

- latences : niveau 4, désignées par le contexte (« the route through object recognition » pour `f1`, qui passe par `C2` ; « the route through rendering » pour `f2`, qui passe par `C4`) ; unité *ms* ;
- `C4` et `C3` : niveau 1, par identifiant, pour faire coexister explicite et implicite dans la même intention ; `C4` en *millicores* (conversion), `C3` en *GB* ;
- style *constraint_statement*, registre *manager*, structure *deux phrases* : latences d'abord, ressources ensuite.

3. Texte

> *Latency can be relaxed a little: up to 511.645 ms for the route through object recognition and up to 506.74 ms for the route through rendering. On resources, C4 needs at least 5150.654 millicores and C3 at least 4.454570016 GB of RAM.*

4. Vérification de chaque valeur

Cible	Écrit	Calcul exact	Canonique	
f1 latence	511.645 ms	511.645×1	511.645 ms	ok
f2 latence	506.74 ms	506.74×1	506.74 ms	ok
C4 cpu	5150.654 millicores	5150.654×0.001	5.150654 vCPU	ok
C3 ram	4.454570016 GB	4.454570016×1	4.454570016 GB	ok

Comparateurs : « up to » → `lte` (deux fois), « at least » → `gte` (deux fois). Direction : « relaxed » est compatible avec 500 → 511.645 et 500 → 506.74. Les identifiants `C4` et `C3` sont présents car leur désignation est de niveau 1 ; `f1` et `f2` sont absents car leur désignation est contextuelle.

5. Sortie attendue de l'extracteur (jamais montrée à l'extracteur ; elle sert de référence)

```
[{"target":"f1","kpi":"e2e_latency","op":"lte","value":511.645,"unit":"ms"},
 {"target":"f2","kpi":"e2e_latency","op":"lte","value":506.74,"unit":"ms"},
 {"target":"C4","kpi":"cpu","op":"gte","value":5.150654,"unit":"vCPU"},
 {"target":"C3","kpi":"ram","op":"gte","value":4.454570016,"unit":"GB"}]
```

Ici l'extracteur doit en outre **convertir** 5150.654 millicores en 5.150654 vCPU et **raisonner** sur les deux périphrases de flux.

5.3 Un refus du contrôle (ARVR-000036)

Le texte contenait « in three *places* ». Le contrôle a refusé : « places » est un mot de placement interdit, même employé au sens courant. Correction du *texte* (« for three stages »), contrôle inchangé, nouvel enregistrement accepté. C'est la règle : le contrôle n'est jamais assoupli.

6 Résultat de la génération

TABLE 6 – Répartition des 315 mentions (calculée sur `intents.json`).

Niveau	Mentions	Part	Cible
1 explicite	51	16 %	20 %
2 fonctionnel	106	34 %	30 %
3 implicite	101	32 %	30 %
4 oblique	57	18 %	20 %

— Indicateurs : 132 latences, 96 RAM, 87 CPU.
— Styles : need_statement 37, narrative 30, constraint_statement 28, imperative 17, goal_oriented 11, ticket_note 11, polite_request 9, question 7.
— Longueur : 7 à 57 mots, médiane 23, moyenne 24,3.
— Unités écrites : ms 100, GB 52, vCPU 55, MB 44, seconds 27, millicores 16, mCPU 8, virtual CPUs 8, autres 5.

6.1 Limites connues

— **Aucune relecture humaine.** Statut de toutes les entrées : `llm_generated_auto_checked`.
— La frontière entre niveaux 3 et 4 n'est pas stricte : « le flux qui se termine par l'overlay » est étiquetée tantôt 3, tantôt 4.
— Quasi-doublon : ARVR-000137 et ARVR-000140 (similarité de mots 0,70).
— Monotonie lexicale partielle : « at least X GB of RAM » revient 16 à 18 fois.
— La longueur croît avec le nombre d'exigences : indice de la taille du scénario, non de la décision.
— **Recouvrement** : le générateur est Claude, l'extracteur évalué est `gpt-5.4-mini` ; aucun recouvrement tant que cela reste vrai, à réévaluer sinon.
— Scénarios synthétiques et stratifiés (35 % STAY, 45 % OFFLOAD, 20 % INFEASIBLE) : ces intentions ne représentent pas des demandes réelles d'opérateurs.
— Pas de GiB/MiB ; pas d'intentions sans seuil chiffré (jeu exploratoire séparé, non produit).

7 Méthode de validation proposée

Le contrôle mécanique garantit l'*exactitude chiffrée*. Il ne dit rien du *sens*. La validation proposée ajoute cinq couches, ordonnées de la moins chère à la plus chère. Seule la couche 0 est faite.

7.1 Couche 1 — Reconstruction à l'aveugle (inspirée de Liu et al.)

Principe : donner à un modèle indépendant l'intention et le contexte applicatif autorisé (rien d'autre), lui demander de reconstruire les exigences, puis comparer à la référence.

1. **Modèle** : au moins deux modèles de familles différentes, ni Claude (générateur) ni l'extracteur évalué, pour éviter la circularité.
2. **Entrée** : champ `intent`, `services.json`, `flows.json`. Jamais `plan`, `mentions`, ni la vérité terrain.
3. **Comparaison** : pour chaque exigence de référence, correspondance exacte sur (cible, indicateur, comparateur, valeur normalisée). Chaque écart est classé : *mauvaise cible*, *mauvaise valeur*, *mauvais comparateur*, *exigence omise*, *exigence ajoutée*.

TABLE 7 – Les couches de validation et leur statut.

	Couche	Ce qu'elle établit	Statut
0	Contrôle mécanique	valeurs, unités, comparateurs, couverture, interdits	**fait**
1	Reconstruction à l'aveugle par LLM indépendant	la cible est-elle retrouvable ?	**proposé, non exécuté**
2	Revue humaine en double aveugle	sens, unicité de la cible, naturel	**proposé, non exécuté**
3	Sous-ensemble rédigé par des humains	absence d'artefact « style LLM »	**proposé, non exécuté**
4	Tests de fuite et d'artefacts	la décision n'est pas devinable depuis le texte	**proposé, non exécuté**
5	Mesures de diversité	pas de quasi-doublons ni de monotonie	**proposé, non exécuté**

4. **Règle de décision** : une intention est **signalée** si au moins un des modèles diverge. Un écart n'est *pas* une preuve d'erreur : il envoie l'intention en couche 2. À l'inverse, un accord ne prouve pas l'unicité de la cible, il la rend plausible.
5. **Lecture des résultats** : le taux d'écart par niveau (1 à 4) doit *croître avec le niveau*. Un écart massif au niveau 1 signale un défaut de protocole ; un écart concentré au niveau 4 sur `f1`/`f2` signale des périphrases ambiguës.

7.2 Couche 2 — Revue humaine en double aveugle

Échantillon. Revue de toute intention qui présente au moins l'un des risques suivants : une mention de niveau 4 (51 intentions), une désignation indirecte de flux de niveau 3 ou 4 (61), trois exigences ou plus (34). Union : **93 intentions**. À cela s'ajoute un tirage aléatoire de 20 % des 57 restantes (environ 12), soit ≈ 105 intentions au total. Toute intention signalée en couche 1 y est ajoutée.

Procédure.

1. Deux annotateurs indépendants reçoivent l'intention, la description des composants et des flux, et **pas** la référence.
2. Pour chaque exigence qu'ils lisent, ils notent cible, indicateur, comparateur, valeur, unité.
3. Ils signalent : (a) toute *ambiguïté de cible* (la phrase conviendrait à plusieurs cibles) ; (b) tout *indice de décision*, c'est-à-dire un ton ou une urgence qui laisse deviner STAY, OFFLOAD ou INFEASIBLE ; (c) un score de *naturel* de 1 (artificiel) à 3 (indiscernable d'un message réel).
4. Un troisième relecteur tranche les désaccords, puis compare à la référence.

Mesures et seuils proposés.

— Accord de Cohen κ entre annotateurs sur l'identification de la cible : viser $\kappa \geq 0{,}8$; en dessous, réviser la grille avant de poursuivre.
— Taux de cible retrouvée par la majorité des annotateurs : viser $\geq 95\,\%$ globalement et aucune intention de niveau 1 ou 2 en échec.
— Toute intention avec ambiguïté de cible ou indice de décision confirmé par deux relecteurs est **réécrite**, puis recontrôlée (couches 0 et 1).

Statuts. `llm_generated_auto_checked` → `semantically_checked` (couches 0 et 1 passées) → `validated` (couche 2 passée).

7.3 Couche 3 — Sous-ensemble rédigé par des humains

Les consignes recommandent une partie du jeu écrite par des humains. Proposition : 30 scénarios choisis au hasard (20 %), rédigés par au moins deux personnes à partir de la même vue masquée (`view`) et des mêmes interdits. On compare l'exactitude de l'extracteur sur ce sous-ensemble et sur les intentions LLM des *mêmes* scénarios. Un écart significatif révèle un artefact de style LLM (trop propre, trop régulier) qui rendrait le jeu plus facile ou plus difficile qu'une demande réelle.

7.4 Couche 4 — Fuite de décision et artefacts

— **Fuite** : entraîner un classifieur simple (régression logistique sur n-grammes, validation croisée) à prédire STAY / OFFLOAD / INFEASIBLE à partir du seul texte. Référence de hasard : classe majoritaire OFFLOAD, soit $68/150 \approx 45{,}3\,\%$. Une exactitude nettement au-dessus indique que le texte trahit la décision.
— **Artefacts** : corrélation longueur–nombre d'exigences (connue, limite documentée) ; association style–indicateur ou style–taille de scénario (doit être faible) ; association ton–niveau de difficulté.

7.5 Couche 5 — Diversité

— Similarité de Jaccard par paire d'intentions : seuil d'alerte 0,6 (la paire 137/140 est à 0,70 et doit être revue).
— Proportion de n-grammes distincts (distinct-2, distinct-3), répartition des mots d'ouverture.
— Fréquence des tournures figées (par exemple « at least X GB of RAM ») : au-delà de 10 % des intentions, reformuler une partie d'entre elles.

7.6 Critères d'acceptation

Le jeu est déclaré utilisable pour une évaluation publiée si :

1. couche 0 à 100 % (déjà vrai) ;
2. couche 1 : toutes les divergences examinées en couche 2 ;
3. couche 2 : $\kappa \geq 0{,}8$, cible retrouvée $\geq 95\,\%$, intentions fautives réécrites et recontrôlées ;
4. couche 4 : pas de fuite détectable ;
5. les limites de la section 6.1 sont reportées dans toute publication.

7.7 Ce que cette validation ne prouve pas

Elle établit que les intentions sont *exactes, non ambiguës et sans fuite*. Elle n'établit pas qu'elles ressemblent à de vraies demandes d'opérateurs : les scénarios sont synthétiques. Cette affirmation exigerait des demandes réelles, hors périmètre.

8 Fichiers livrés

Fichier (racine du dépôt)	Rôle
`intents.json`	150 intentions avec `plan`, `mentions`, `review_status`
`validation_report.json`	résultat des contrôles, rejets, répartitions, limites
`intent_generation_report.md`	rapport demandé par les consignes
`SKILL.md`	procédure suivie (protocole par scénario, reprise)
`PROMPT_GENERATION_INTENTS.md`	prompt de travail (rôle, niveaux, interdits, format)
`intent_store.py`	affichage masqué, contrôle, enregistrement atomique (n'écrit aucun texte)
`methodologie_generation_intents.pdf`	ce document

Utilisation sans fuite. Donner à l'extracteur *uniquement* le champ `intent` et le contexte applicatif autorisé. Les champs `plan`, `mentions`, `baseline_mentions` et `n_words` sont des métadonnées d'analyse. La vérité terrain n'est lue qu'après la prédiction, via `scenario_id`.